Day 3 Deliverable: Processing Steps, Algorithm Configuration, Outputs and Interface Integration

Insider Threat Detection System, Capstone Group 7 · 5 November 2026

1. Purpose

This document is the Day 3 deliverable. It answers both columns of the task table.

Column	Day 3 task	Where answered
Research and methodology	Describe data-processing steps, algorithm configuration and output generation	Sections 2, 3 and 4
Prototype development	Integrate the main processing module with the system interface	Sections 5 and 6

Evidence is in Section 7. Limits and open items are in Section 8. All values below were read from the code of the repository, not from earlier text.

2. Data-processing steps

The detection engine (itd/engine.py) is a batch process. It runs the steps in this order.

Step	What happens	Main rules
0. Input validation	The data folder is checked before anything is read (Day 2, itd/validate.py).	Errors stop the run; warnings are reported and the run continues.
1. Ingestion	The five event files (logon, device, email, file, http) and answers.csv are read in chunks. Per-session counters are built from them.	Chunks of 500,000 rows. Extra columns are ignored.

Step	What happens	Main rules
2. Sessions	One session per user and PC. A session starts at a Logon and ends at the matching Logoff. A missing logoff closes the session at the next Logon or after 12 hours, whichever comes first. Sessions that cross midnight are kept whole. Each event is attached to the session of the same user and PC that contains it.	SESSION_MAX_HOURS = 12. After hours means before 07:00 or after 19:00.
3. Features and baselines	39 features per session: 28 session features, 10 deviation features and 1 recency feature. For 10 metrics each user has a running mean and standard deviation.	Details below.
4. Models	Three models are trained once on the training part, saved, and loaded for scoring.	Section 3.
5. Risk score and explanation	Calibrated scores are combined into a 0 to 100 risk score. Sessions at or above the threshold are flagged. SHAP and baseline deviations explain each session.	Section 3.
6. Storage	Results, baselines, models and the run record are written to MongoDB, and metrics.json and validation.json are written to the report folder.	Section 4.

2.1 Baselines and deviations

- The 10 baseline metrics are: logon hour, session length, removable-device connections, distinct web pages, file events, archive files, emails sent, emails sent outside the company, email size and file-sharing or leak site visits.

- Each user has a running mean and standard deviation (Welford method). A session is scored against the baseline as it was before that session, and the baseline is updated afterwards, so a session never sees its own data.
- A deviation is the difference from the user's mean divided by the user's standard deviation, clipped to the range -10 to +10. The standard deviation has a floor of 0.5, so a metric the user never used still gives a finite value.
- A deviation is trusted only after 5 earlier sessions of that user (MIN_HISTORY = 5); before that it is 0.
- The recency feature is the number of working hours since the user's previous session. Whole weekend days are subtracted, so a normal Friday-to-Monday gap is not read as a long absence.
- A saved baseline is used only when the new batch continues after the user's last saved session. If the batch starts at or before that time (for example the same data again), the baseline is rebuilt from the batch.

2.2 Train, validation and test parts

Sessions are sorted by start time. The first 70% are used for training, the next 15% for validation and the last 15% for testing (SPLIT = 0.70, 0.15, 0.15). A random split is not used, so the models are never trained on sessions that happened after the sessions they are tested on. The engine stops if any of the three parts has no malicious session. When a saved model is reused without retraining, the sessions are labelled scored instead of train, validation or test.

3. Algorithm configuration

Component	Configuration
Logistic Regression (baseline)	StandardScaler followed by LogisticRegression with class_weight balanced and max_iter 2000
XGBoost (main model)	200 trees, max_depth 4, learning_rate 0.1, subsample 0.9, scale_pos_weight equal to negatives divided by positives in the training part, eval_metric aucpr, random_state 42, n_jobs 2
Isolation Forest (unsupervised)	200 trees, contamination auto, random_state 42, n_jobs 2. It does not use labels. Its score is the negative of score_samples, scaled to 0 to 1 with the minimum and maximum seen in training and clipped.

Component	Configuration
Calibration	One Platt curve per model, p = 1 / (1 + exp(-(a × score + b))), fitted with logistic regression (C = 10) on the validation part only
Model weights	Each model's best validation F1 divided by the sum of the three, so the weights add up to 1
Risk score	100 × the weighted sum of the three calibrated scores, rounded to one decimal
Flag threshold	The score that gives the best F1 of the combined score on the validation part. A session is flagged when its score is at or above it.
Test part	Used only to report precision, recall, F1, average precision and the Brier score. No threshold or weight is tuned on it.
Randomness	One fixed seed (RANDOM_STATE = 42)
Storage of models	Saved in MongoDB GridFS with a SHA-256 hash for each model. Loading stops with an error if a hash does not match. --retrain trains new models and rebuilds the baselines.

Where to change a value: session and split settings are in itd/config.py, the baseline settings in itd/baselines.py, the calibration constant in itd/risk.py and the model settings in itd/models.py. The hyperparameters are fixed in the code. The project does not tune them.

4. Output generation

For every session the engine stores one document in the results collection:

Field	Content
session_id, run_id, user, pc, start, end	Identity of the session and the run that produced it
risk_score	0 to 100, one decimal
flagged	true when the score is at or above the threshold
model_scores	The three calibrated scores (lr, xgb, iforest), four decimals

Field	Content
top_shap	Up to 3 features that pushed the XGBoost score up the most. Each has the feature name, value, SHAP value, a strength (strong when it is at least 25% of the largest) and a sentence in plain words.
baseline_deviations	Up to 3 metrics with the largest positive deviation: value, user mean, user standard deviation, deviation and a sentence in plain words
label, split	The known label and the part of the time split (train, validation, test or scored)

The SHAP explanation covers the XGBoost model only, not the combined score. It is computed in blocks of 50,000 sessions.

Other outputs:

Output	Content
runs collection	One record per run: seconds, seconds per step, number of sessions and flagged sessions, threshold, weights, test metrics and the validation summary
user_baselines collection	The saved baseline of each user, so the next batch continues from it
trained_models collection and GridFS	The three models, hashes, calibration curves, weights and thresholds
reports/metrics.json	The run record and the test metrics
reports/validation.json	The full input-validation report

The dashboard reads these outputs through the API: the flagged list (sorted by risk, with CSV download), the session detail with its explanation, the user page and the run panel with the test results.

5. Prototype: connecting the processing module to the interface

Before Day 3 the engine could only be started from the command line. The dashboard can now request a run and follow it.

5.1 Design

The security design allows the API to write only the access log, and only the engine account writes results. A button that starts the engine inside the API would break that rule. So the dashboard files a request and a separate worker performs it.

1. An administrator chooses a dataset folder and, if wanted, Train new models, then clicks Start run.
2. The API checks the role and the folder name, stores a request with the status requested and writes a run_requested entry in the access log. It answers 202.
3. The worker (python -m itd.worker, engine database account) takes the oldest waiting request and sets it to running.
4. The worker runs the engine (steps 0 to 6). The engine output is kept as a log.
5. The worker records done or failed, the end time, the log tail, the reason when it failed and the id of the run record.
6. While a run is active the dashboard asks for the status every 3 seconds, otherwise every 20 seconds. When a run finishes, the flagged list loads again.

Statuses: requested, running, done, failed. Only one run is active at a time.

5.2 API

Endpoint	Role	Purpose	Errors
POST /api/run	admin	File a run request (dataset folder name, retrain)	401 no token, 403 not admin, 404 dataset not found, 409 a run is already waiting or running, 422 bad input
GET /api/run/status	analyst or admin	The newest request and its status. Only an admin also receives the log text.	401
GET /api/run/datasets	admin	Folder names under the data folder that contain logon.csv	401, 403
POST /api/login	any	Now also returns the role, so the dashboard shows the controls to administrators only	401, 429

5.3 Security decisions

- Only administrators can start a run. Analysts stay read-only and see a note while a run is in progress.
- The dashboard sends a folder name, never a path. The name must match a strict pattern and the folder must contain logon.csv inside the data folder (ITD_DATA_ROOT, default data). Names such as ../x or a/b are rejected.
- The API account may only read and insert run requests and insert access-log entries. The worker uses the engine account to update progress. The API still cannot write results.
- A request is stored with a database validation rule (status, folder name pattern, maximum lengths).
- When the worker starts, any request still marked running belongs to a worker that stopped, and it is marked failed.
- A failed run keeps the earlier results (Day 1 change P1), and bad input stops the run before any data is read (Day 2).

6. Code changes on Day 3

File	Change
backend/itd/jobs.py	New. Request, status, claim and process logic.
backend/itd/worker.py	New. The worker program.
backend/itd/api.py	Three endpoints, a request model, and the role in the login answer
backend/itd/security.py	require_admin
backend/itd/schema.py	New collection run_requests with rules and indexes; new log action run_requested
backend/itd/engine.py	Returns the run id so a request can point to its run record
backend/itd/config.py	DATA_ROOT (environment variable ITD_DATA_ROOT)
mongo-init/init-users.js	The API account may find and insert in run_requests
frontend/src/components/RunControl.jsx	New. Dataset choice, retrain option, Start run, live status, failure reason and details

File	Change
frontend/src/api.js, App.jsx	Role, three API calls, the run panel, and a list reload when a run ends
backend/tests/test_run_control.py	New. 12 tests

The tests check: admin-only access, the stored and logged request, one active run, bad folder names, the dataset list, an empty status, the role in login, a finished run, failures with their reasons, the worker loop and stale requests, the schema, and a full path from request through the API, the worker and the real engine to results read through the API.

7. Evidence

Figure 1 shows the new backend files jobs.py and worker.py being created and compiling.

```
>>     def __init__(self, *streams):
>>         self.streams = streams
>>
>>     def write(self, s):
>>         for st in self.streams:
>>             st.write(s)
>>         return len(s)
>>
>>     def flush(self):
>>         for st in self.streams:
>>             st.flush()
>>
>>
>> def process(db, doc, runner):
>>     """Run one claimed request with runner(path, retrain) and record how it ended. Never raises."""
>>     buf = io.StringIO()
>>     status, message, run_id = "done", None, None
>>     try:
>>         with contextlib.redirect_stdout(_Tee(sys.stdout, buf)):
>>             info = runner(dataset_path(doc["dataset"]), doc["retrain"])
>>         run_id = info.get("run_id") if isinstance(info, dict) else None
>>     except SystemExit as e:                                  # the engine stops this way on bad input
>>         status, message = "failed", str(e.code) if isinstance(e.code, str) else "The run stopped before it finis
d"
>>     except Exception as e:
>>         status, message = "failed", f"{type(e).__name__}: {e}"
>>     fields = {"status": status, "finished_at": time.time(), "log_tail": buf.getvalue()[-LOG_CHARS:]}
>>     if message:
>>         fields["message"] = message[:MESSAGE_CHARS]
>>     if isinstance(run_id, ObjectId):
>>         fields["run_id"] = run_id
>>     db.run_requests.update_one({"_id": doc["_id"]}, {"$set": fields})
>>     return status
>> '@ | Set-Content -Encoding UTF8 itd\jobs.py
PS C:\Users\f\Desktop\itd_clean\backend>
PS C:\Users\f\Desktop\itd_clean\backend> @'
>> """Run worker. Performs the batch runs that the dashboard requests. Start it next to the API:
>>     python -m itd.worker              keep running, check for requests every 3 seconds
>>     python -m itd.worker --once       handle the waiting requests, then stop
>> Uses the engine database account (MONGO_URI_ENGINE), so the API account never needs write rights to results."""
>> import argparse
>> import time
>>
>> from . import jobs
>> from .db import get_db
>>
>>
>> def serve(poll=3.0, once=False, runner=None):
>>     if runner is None:
>>         from . import engine                                 # heavy imports only when a real run is needed
>>         runner = lambda path, retrain: engine.run(path, retrain=retrain)
>>     db = get_db("engine")
>>     stale = jobs.recover_stale(db)
>>     if stale:
>>         print(f"Marked {stale} unfinished request(s) from an earlier worker as failed.")
>>     print(f"Worker ready (data root: {jobs.config.DATA_ROOT}). Waiting for run requests ...")
>>     while True:
>>         doc = jobs.claim_next(db)
>>         if doc:
>>             print(f"Run requested by {doc['requested_by']}: dataset '{doc['dataset']}', retrain={doc['retrain']}
>>             print("Finished:", jobs.process(db, doc, runner))
>>         elif once:
>>             return
>>         else:
>>             time.sleep(poll)
>>
>>
>> if __name__ == "__main__":
>>     ap = argparse.ArgumentParser()
>>     ap.add_argument("--poll", type=float, default=3.0)
>>     ap.add_argument("--once", action="store_true")
>>     a = ap.parse_args()
>>     try:
>>         serve(a.poll, a.once)
>>     except KeyboardInterrupt:
>>         print("Worker stopped.")
>> '@ | Set-Content -Encoding UTF8 itd\worker.py
PS C:\Users\f\Desktop\itd_clean\backend> python -m py_compile itd\jobs.py itd\worker.py; "jobs.py and worker.py OK"
jobs.py and worker.py OK
PS C:\Users\f\Desktop\itd_clean\backend>
```

Figure 1. Creation of the run-request logic and the worker.

Figure 2 shows the backend patch applied to six files (config, security, api, schema, engine and the database user script), followed by a successful compile.

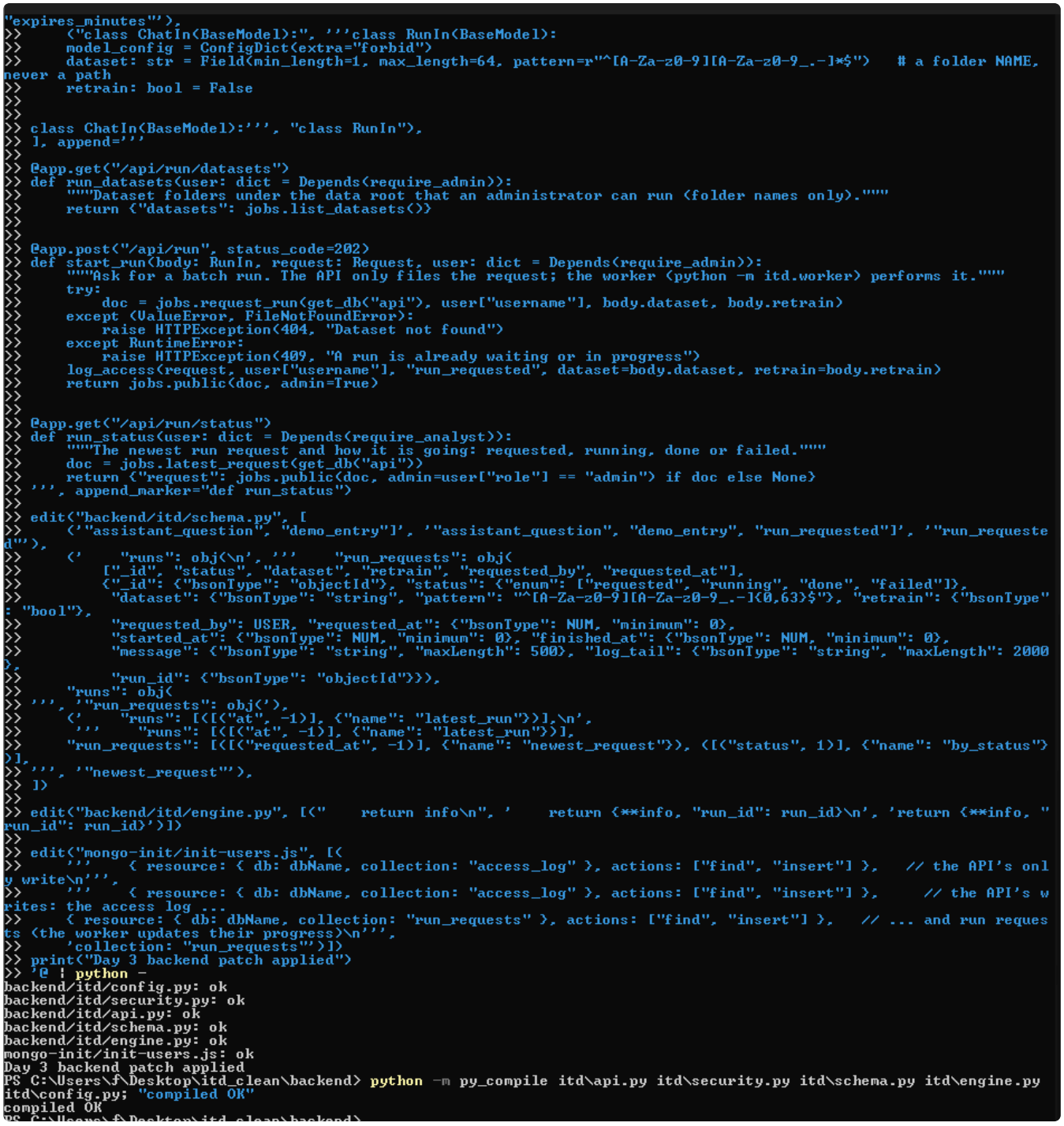


Figure 2. The backend patch.

Figure 3 shows the dashboard patch applied to api.js and App.jsx, followed by a successful production build (42 modules transformed).

```
>> def edit(name, steps, append=None, append_marker=None):
>>     p = os.path.join(src, name)
>>     s = open(p, newline="", encoding="utf-8").read()
>>     crlf = "\r\n" in s
>>     s = s.replace("\r\n", "\n")
>>     for anchor, new, marker in steps:
>>         if marker in s:
>>             print(f"  {name}: already applied ({marker[:35]})")
>>             continue
>>         assert s.count(anchor) == 1, f"{name}: anchor not found exactly once: {anchor[:60]!r}"
>>         s = s.replace(anchor, new, 1)
>>     if append and append_marker not in s:
>>         s = s.rstrip("\n") + "\n" + append
>>     open(p, "w", newline="", encoding="utf-8").write(s.replace("\n", "\r\n") if crlf else s)
>>     print(name + ": ok")
>>
>> edit("api.js", [
>>     ('export const getUsername = () => sessionStorage.getItem("username") || "";\n',
>>      'export const getUsername = () => sessionStorage.getItem("username") || "";\nexport const getRole = () => ses
sionStorage.getItem("role") || "";\n', "export const getRole"),
>>     ('sessionStorage.removeItem("username"); }', 'sessionStorage.removeItem("username"); sessionStorage.removeItem
("role"); }', 'removeItem("role")'),
>>     ('  if (status === 422)', '  if (status === 409) return "A run is already waiting or in progress.";\n  if (sta
tus === 422)', "status === 409"),
>>     ('  sessionStorage.setItem("username", r.username);\n', '  sessionStorage.setItem("username", r.username);\n
sessionStorage.setItem("role", r.role || "");\n', 'setItem("role"'),
>> ], append='''export const getRunStatus = () => request("/api/run/status");
>> export const getRunDatasets = () => request("/api/run/datasets");
>> export const startRun = (dataset, retrain) => request("/api/run", { method: "POST", body: JSON.stringify({ dataset
, retrain }) });
>> ''', append_marker="export const startRun")
>>
>> dot = "\u00b7"
>> edit("App.jsx", [
>>     ('import { getUsername, isLoggedIn, logout } from "./api.js";', 'import { getRole, getUsername, isLoggedIn, lo
gout } from "./api.js";', "getRole, getUsername"),
>>     ('import Logo from "./Logo.jsx";', 'import Logo from "./Logo.jsx";\nimport RunControl from "./components/RunCo
ntrol.jsx";', "import RunControl"),
>>     ('  const [notice, setNotice] = useState("");\n',
>>      '  const [notice, setNotice] = useState("");\n  const [reload, setReload] = useState(0);          // changes
when a run finishes, so the list loads again\n', "setReload] = useState"),
>>     ("{getUsername()} " + dot + " Analyst", "{getUsername()} " + dot + ' {getRole() === "admin" ? "Administrator"
: "Analyst"}', 'getRole() === "admin"'),
>>     ("          : <FlaggedSessions />}", "          : <><RunControl onFinished={() => setReload((n) => n + 1)} /><
FlaggedSessions key={reload} /></>}", "<RunControl onFinished"),
>> ])
>> print("Day 3 frontend patch applied")
>> '@ | python -
api.js: ok
App.jsx: ok
Day 3 frontend patch applied
PS C:\Users\f\Desktop\itd_clean\frontend\src> cd ..
PS C:\Users\f\Desktop\itd_clean\frontend> npm install

added 63 packages, and audited 64 packages in 8s

7 packages are looking for funding
  run `npm fund` for details

2 vulnerabilities (1 moderate, 1 high)

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
npm notice
npm notice New major version of npm available! 10.9.2 -> 12.2.2
npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.2.2
npm notice To update run: npm install -g npm@12.2.2
npm notice
PS C:\Users\f\Desktop\itd_clean\frontend> npm run build

> itd-dashboard@0.1.0 build
> vite build

vite v5.4.21 building for production...
✓ 42 modules transformed.
dist/index.html                   0.64 kB │ gzip:  0.39 kB
dist/assets/index-DtfHKjG5.css    5.46 kB │ gzip:  1.79 kB
dist/assets/index-DxMqv9NJ.js   168.31 kB │ gzip: 53.86 kB
✓ built in 748ms
```

Figure 3. The dashboard patch and the production build.

Figure 4 shows the test run: 63 passed, 1 skipped. That is the 51 earlier tests plus the 12 new ones, in 52.97 seconds.

```
PS C:\Users\f\Desktop\itd_clean\frontend> cd "$env:USERPROFILE\Desktop\itd_clean\backend"
PS C:\Users\f\Desktop\itd_clean\backend> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\f\Desktop\itd_clean\backend> python -m pytest -q
.......................s.......................................                                [100%]
=============================================== warnings summary ===============================================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\f\Desktop\itd_clean\backend\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
63 passed, 1 skipped, 1 warning in 52.97s
(.venv) PS C:\Users\f\Desktop\itd_clean\backend>
```

Figure 4. Test result: 63 passed, 1 skipped.

Live run on a real MongoDB, 6 November 2026. The database rules were applied with python -m itd.schema apply, then the API, the worker and the dashboard were started in three windows. The accounts boss (administrator) and alice (analyst) were used.

Figure 5 shows the administrator view. The header reads boss · Administrator. The Run analysis card shows a finished run of cert_synthetic (asked at 7:57:17, ended 7:57:29), and the flagged list was loaded again by itself: 66 flagged of 7,329 sessions, threshold 19.3.

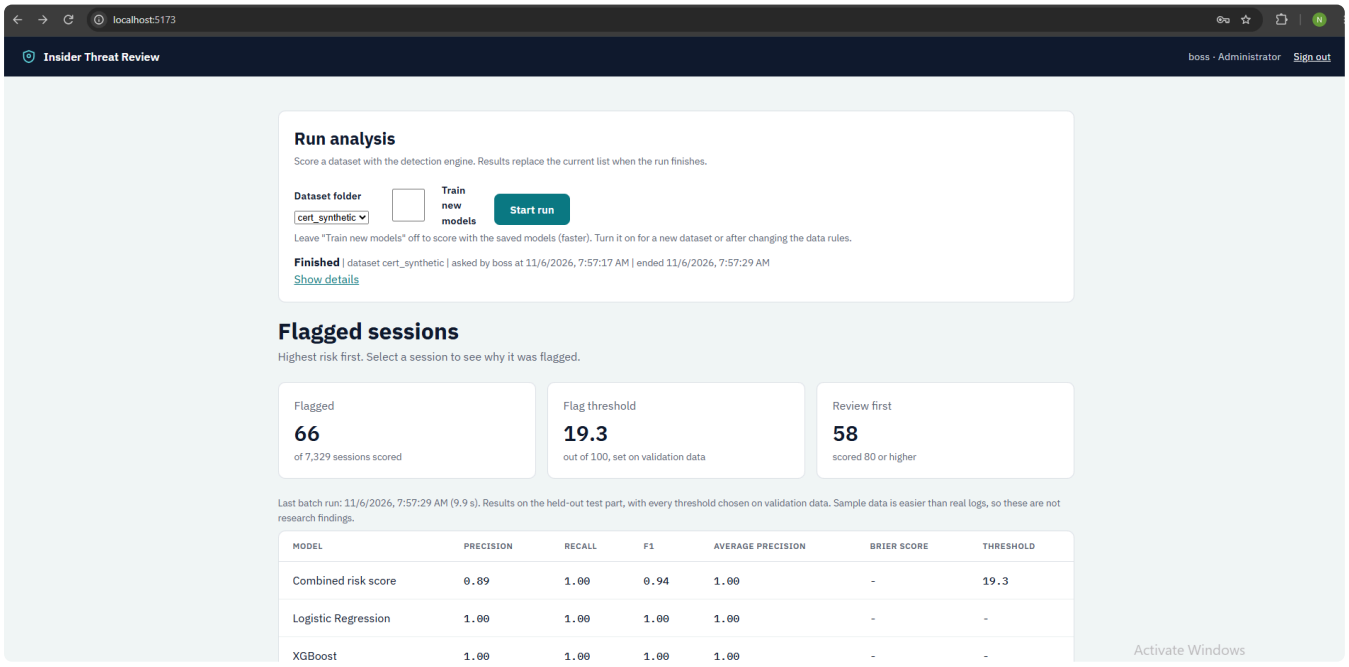


Figure 5. Administrator view after a finished run.

Figure 6 shows the analyst view of the same data. The header reads alice · Analyst. The same list and test metrics are visible, and there is no Run analysis card.

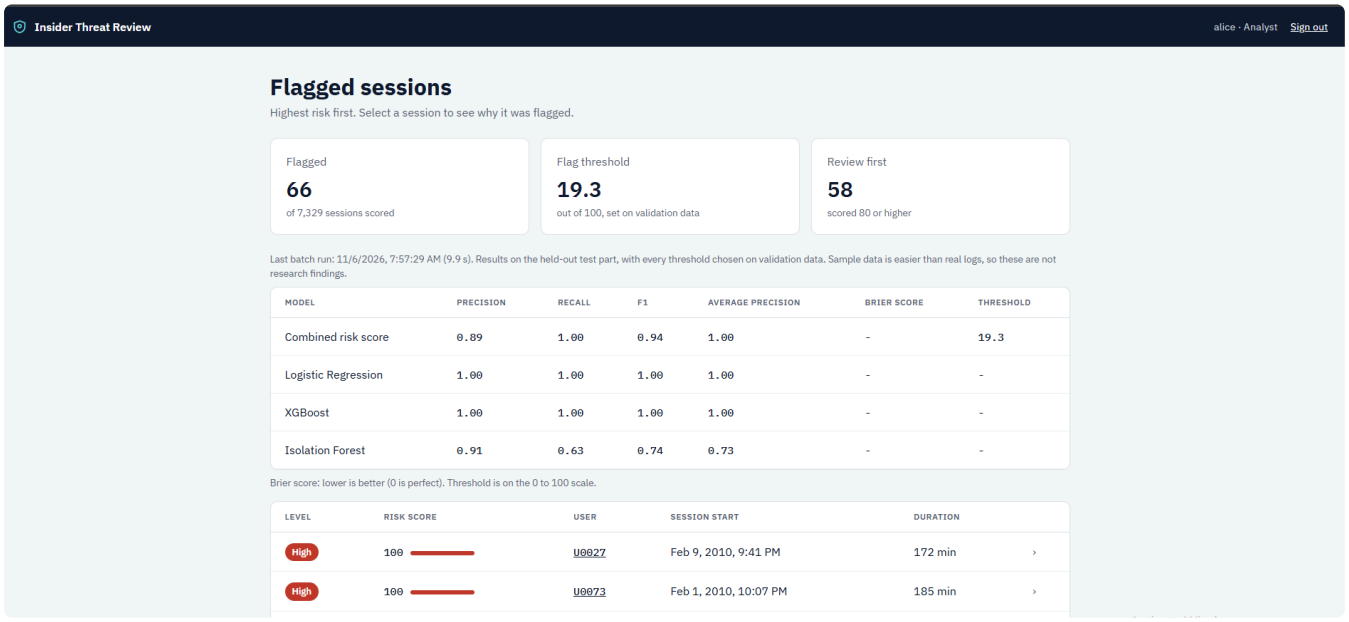


Figure 6. Analyst view: read-only, no run controls.

Figure 7 shows a run of the dataset cert_broken, a copy of the data with device.csv removed. It ends as Failed after 2 seconds with the message Input validation failed.

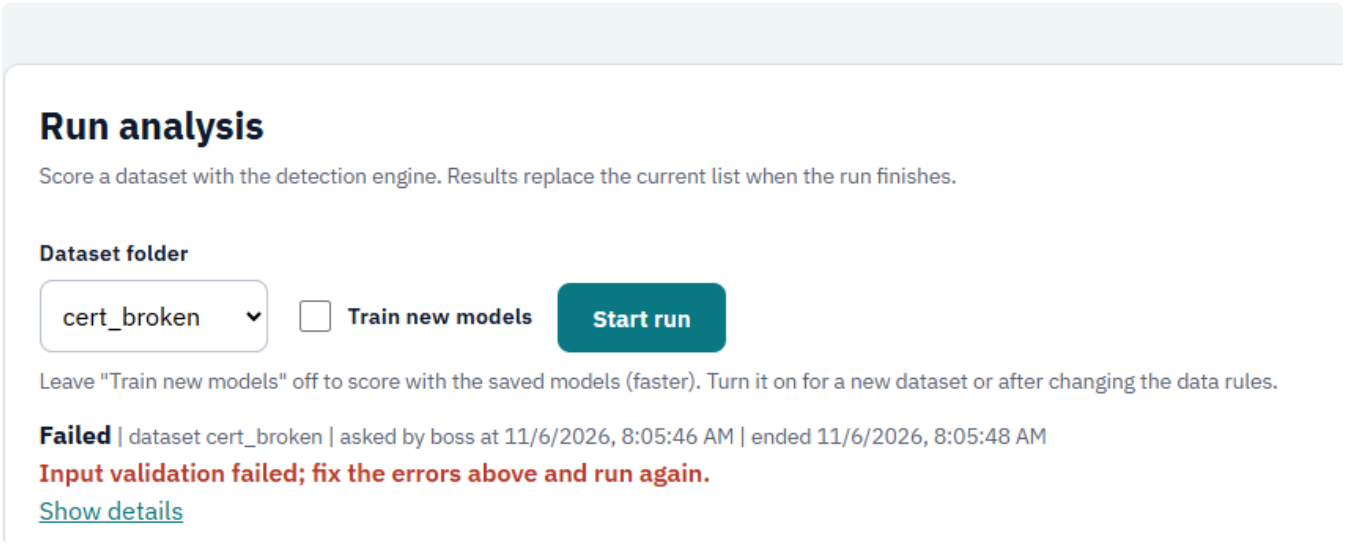


Figure 7. A rejected dataset.

Figure 8 shows the details of that run: the validation line [V1 error] device.csv: a required CSV file is missing. The flagged list of the earlier run stayed unchanged.

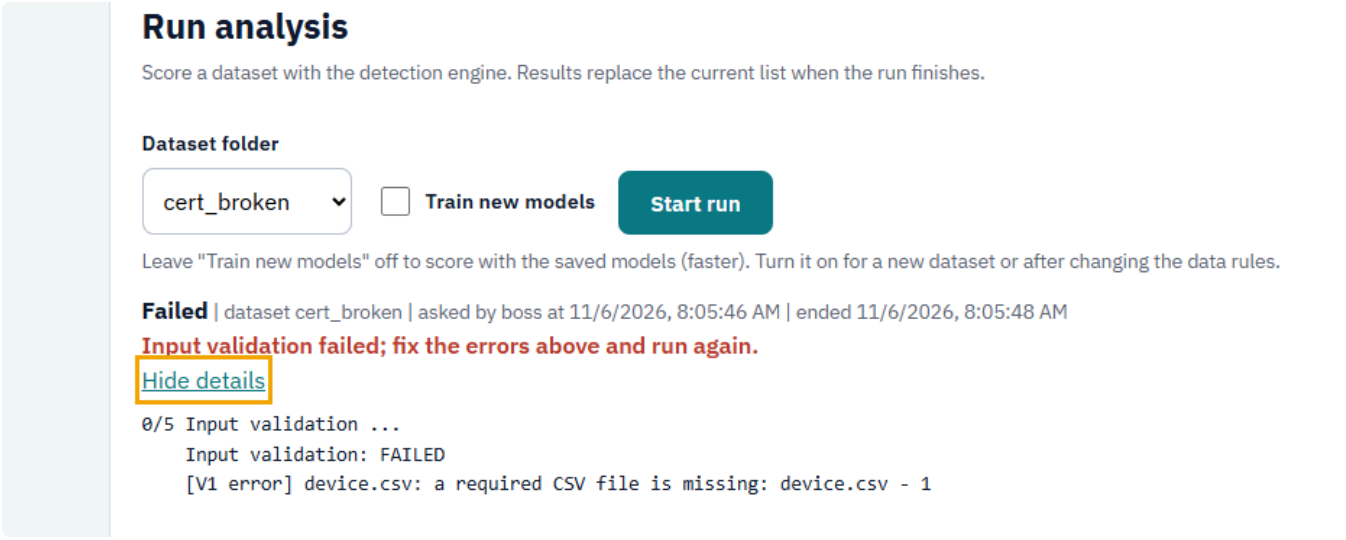


Figure 8. Details of the failed run.

Records stored after the successful run, read from MongoDB (shortened):

```
results by split: [{'_id': 'scored', 'n': 7329}]
request: status done, dataset cert_synthetic, retrain False, requested_by
boss, run_id linked
run validation: {'status': 'ok', 'errors': 0, 'warnings': 0, 'rows':
{'logon': 14440, 'device': 290, 'email': 23943, 'file': 18910, 'http': 51052,
'answers': 681}}
log entry: action run_requested by boss, dataset cert_synthetic, retrain
False
```

8. Limits and open items

- The run from the dashboard has been tested with the test database (including one full path through the real engine). This was shown live on 6 November 2026 against a real MongoDB with the API, the worker and the dashboard running (Figures 5 to 8). The option Train new models was not run live.
- The worker must be running. If it is not, the request stays waiting and the dashboard says so after 20 seconds.
- One worker runs one request at a time. The check for an active run is best effort: two requests at the same instant can both queue, and the worker runs them one after the other.
- The dashboard checks the status by asking at intervals. It does not receive pushed updates.
- The saved log keeps the last 1,500 characters of the engine output.
- In a MongoDB set up with separate accounts, the init script only runs on an empty database. An existing database needs its API role updated and python -m itd.schema apply run once.
- Runs have not been timed on a large release of CERT r4.2. The model settings are fixed and not tuned, and all results so far come from generated data.